

Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System

FreeRTOS on STM32 Blue Pill, simulated in Wokwi

Last Name	Mendoza	Date	September 27, 2026
First Name	Cassandra Khan P.	Subject/Section	BCA182 / B187.1
Repository	github.com/monxx-ie/BCA182-freetos-multisensor	Instructor	Asst. Prof. Paul Rodolf P. Castor

Summary

This report documents the design, implementation and verification of a simulated room-monitoring device built on an STM32F103C8 (Blue Pill) with PlatformIO, the STM32Cube HAL and native FreeRTOS APIs. Six tasks share the work: sensing (DHT22 and photoresistor), display (SSD1306 OLED), input (rotary encoder), motion monitoring (PIR), temperature alarm (buzzer) and an ACTIVE/INACTIVE state machine. Tasks communicate through three queues, one event group and one mutex. The build succeeds, all 13 unit tests pass, *pico check* reports 0 high and 0 medium findings, and all ten functional tests (FT-01 to FT-10) pass in Wokwi. A significant part of the work was diagnosing why the standard FreeRTOS Cortex-M ports do not run in the Wokwi STM32F103 model; this is described in Section 7.

1. Problem and Requirements

A room monitor must sample environmental conditions continuously, react to people (motion and user input) with low latency, warn when the temperature is outside a safe range, and stop doing unnecessary work when the room is empty. These activities run at different rates and have different urgency, which makes the problem a good fit for a preemptive RTOS: each activity becomes a task that blocks until it has work, and the scheduler decides which ready task runs.

Functional requirements

ID	Requirement	How it is met
FR-01	Temperature measurement	SensorTask reads the DHT22 every 2 s
FR-02	Humidity measurement	SensorTask reads the DHT22 every 2 s
FR-03	Ambient-light measurement	SensorTask reads ADC1 channel 0, converted to a relative 0-100 %
FR-04	Motion detection	MotionTask polls the PIR every 100 ms
FR-05	OLED display	DisplayTask shows one selected measurement
FR-06	Encoder navigation	InputTask: Temperature - Humidity - Light - Motion, both directions, wraparound
FR-07	Temperature alarm	AlarmTask: buzzer on below 18 C or above 30 C
FR-08	Activity state	StateTask holds ACTIVE / INACTIVE, published as EVENT_ACTIVE
FR-09	Automatic inactivity	15 s without motion changes the state to INACTIVE
FR-10	Automatic reactivation	Any motion returns the system to ACTIVE

Wokwi adaptation

All parts are Wokwi simulation models: board-stm32-bluepill, wokwi-dht22, wokwi-photoresistor-sensor, wokwi-pir-motion-sensor, wokwi-ky-040, board-ssd1306 and wokwi-buzzer. Test stimuli are applied through each part's control panel (temperature and humidity sliders, lux slider, *Simulate motion*, encoder arrows). The serial monitor is wired to USART1 (PA9/PA10). Two adaptations were needed because of simulator limits: the DHT22 uses the DWT cycle counter for microsecond timing (DWT is supported, DMA and IWDG are not), and FreeRTOS runs on a Wokwi compatibility port (Section 7). The light reading is a relative percentage, not calibrated lux.

2. System Architecture and Design

Hardware architecture

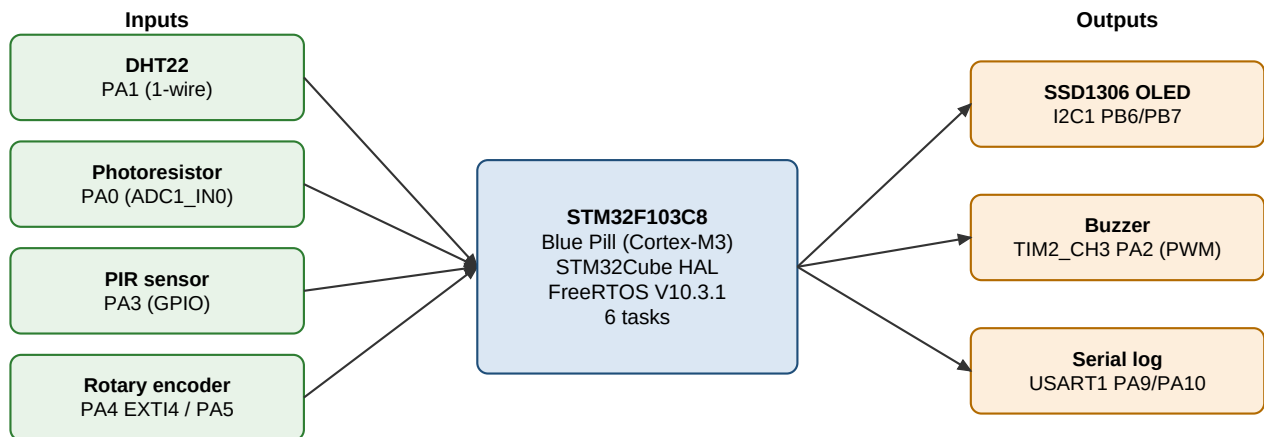


Figure 1. Hardware architecture: four inputs, the STM32F103C8 running FreeRTOS, three outputs.

Signal	Pin	Peripheral / mode	Reason for choice
Photoresistor AO	PA0	ADC1_IN0, analog	PA0-PA7 are ADC-capable
DHT22 data	PA1	GPIO: output for start pulse, then input + pull-up	Moved off PA0 to free the ADC pin; STM32F1 pull-ups only work in input mode
Buzzer	PA2	TIM2_CH3 PWM, 1 kHz	PB0 was TIM3, which became the RTOS tick
PIR	PA3	GPIO input, pull-down	Idle low when no motion
Encoder CLK / DT	PA4 / PA5	EXTI4 falling edge / GPIO input	Edge interrupt catches every detent
USART1 TX / RX	PA9 / PA10	115200 8N1	Wokwi serial monitor
OLED SCL / SDA	PB6 / PB7	I2C1, 400 kHz	Default I2C1 pins
Status LED	PC13	GPIO output	Assert / fault indicator

Timers: TIM2 for buzzer PWM, TIM3 for the FreeRTOS tick (compatibility port), TIM4 as the HAL timebase generated by STM32CubeMX, so HAL_GetTick() does not compete with the RTOS for SysTick.

Software architecture and subsystem decomposition

The firmware is split into small modules. **Drivers** talk to hardware only (dht22.c, ldr.c, ssd1306.c, buzzer.c, log.c). **Task modules** each own one responsibility (sensors.c, display.c, input.c, alarm.c, motion.c, system_state.c). **logic.c** holds pure decision functions with no HAL or FreeRTOS calls, which is what makes them unit-testable on a PC. **rtos_objects.c** creates and documents every queue, the event group and the mutex. **main.c** only initializes hardware and calls **app_main()** in app.c, which creates the RTOS objects and tasks and starts the scheduler.

Layer	Files	Depends on
Entry	main.c, app.c	HAL init, then FreeRTOS objects and tasks
Tasks	sensors.c, display.c, input.c, alarm.c, motion.c, system_state.c	drivers, logic, rtos_objects
Pure logic	logic.c	nothing (standard C only)
Drivers	dht22.c, ldr.c, ssd1306.c, buzzer.c, log.c	STM32 HAL / registers
RTOS glue	rtos_objects.c, rtos_hooks.c, FreeRTOSConfig.h	FreeRTOS
Generated	stm32f1xx_hal_msp.c, stm32f1xx_it.c, stm32f1xx_hal_timebase_tim.c	STM32CubeMX

State machine

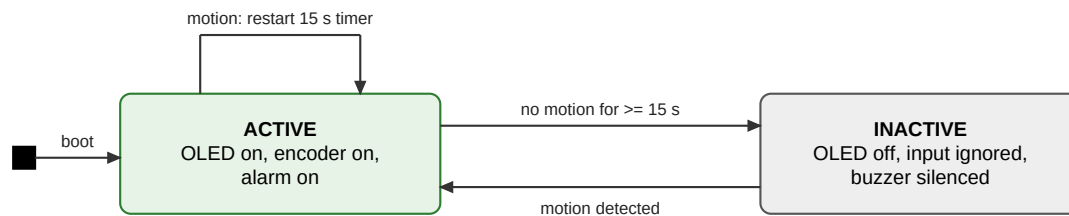


Figure 2. ACTIVE/INACTIVE state machine, implemented by the pure function `evaluateSystemState()`.

In ACTIVE the OLED is on, sensor processing continues, the encoder changes pages and the alarm can sound. In INACTIVE the OLED is switched off and `DisplayTask` blocks on `EVENT_ACTIVE`, so it performs no display work at all; encoder steps are discarded and the buzzer is forced off. `MotionTask` keeps running in both states because motion is what wakes the system.

3. FreeRTOS Architecture

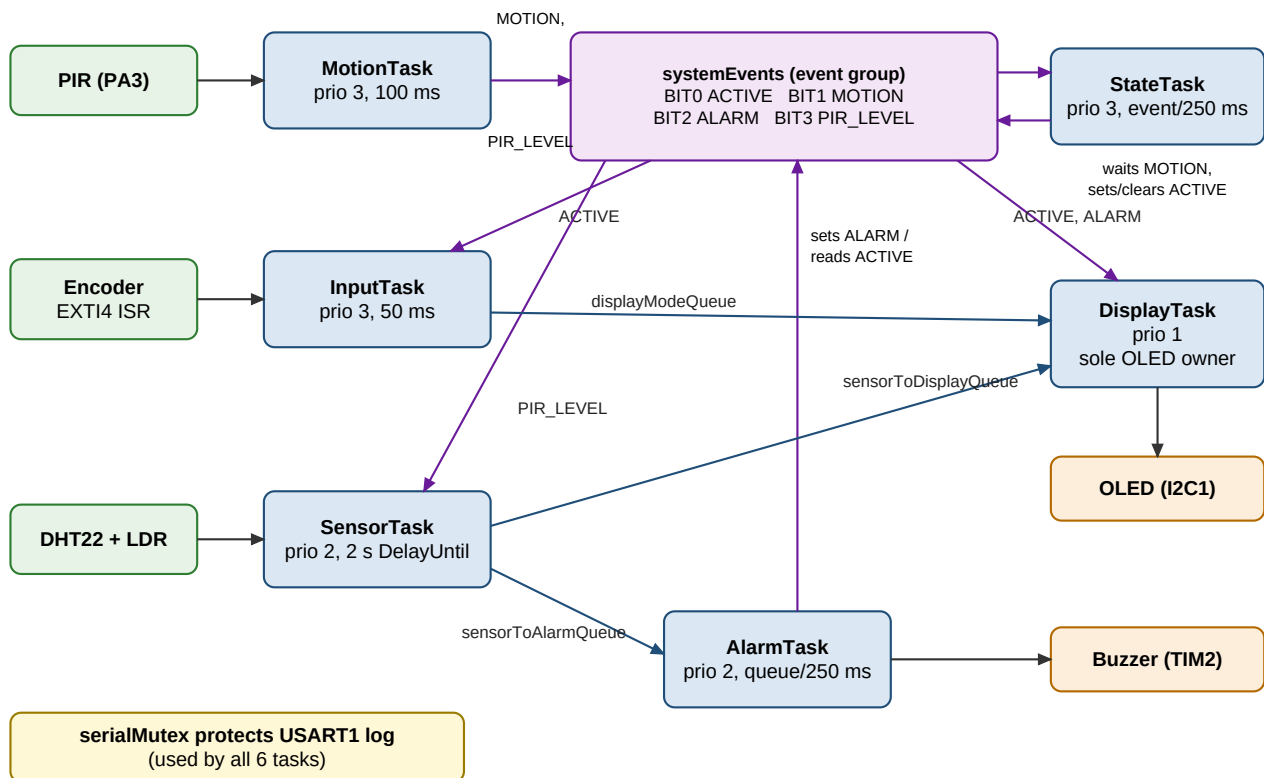


Figure 3. Task communication: blue arrows are queues, purple arrows are event-group bits, and every task logs through `serialMutex`.

Task table

Task	Responsibility	Trigger / period	Prio	IPC	Typical blocked condition
MotionTask	Read PIR, signal motion	100 ms	3	Sets <code>EVENT_MOTION</code> , <code>EVENT_PIR_LEVEL</code>	<code>vTaskDelay(100 ms)</code>
InputTask	Encoder steps to page	50 ms + EXT14 counter	3	<code>displayModeQueue</code> ; reads <code>EVENT_ACTIVE</code>	<code>vTaskDelay(50 ms)</code>
StateTask	ACTIVE/INACTIVE	Motion event or 250 ms	3	Waits <code>EVENT_MOTION</code> ; sets/clears <code>EVENT_ACTIVE</code>	<code>xEventGroupWaitBits()</code>
SensorTask	DHT22 + LDR	2 s	2	Overwrites both sensor queues; reads <code>EVENT_PIR_LEVEL</code>	<code>vTaskDelayUntil()</code>

Task	Responsibility	Trigger / period	Prio	IPC	Typical blocked condition
AlarmTask	Evaluate temperature, buzzer	New reading or 250 ms	2	sensorToAlarmQueue; sets/clears EVENT_ALARM	xQueueReceive()
DisplayTask	Sole OLED owner	New data/page or 100 ms	1	sensorToDisplayQueue, displayModeQueue, event group	xQueueReceive(); INACTIVE: WaitBits(ACTIVE)

Priority justification

Priorities are based on how much latency each task can tolerate, not on how important the feature is.

MotionTask, InputTask and StateTask (3) respond to people. If a motion edge is handled late the system stays asleep while someone is in the room, and an encoder that lags behind the user's hand feels broken. Their work per activation is tiny (read a pin, update a counter or a bit), so running them first costs other tasks almost nothing. **SensorTask and AlarmTask (2)** work on a 2-second cadence; a delay of a few milliseconds is invisible for room temperature. **DisplayTask (1)** does the longest operation, a full 1 KB I2C frame transfer, and a slightly late redraw is harmless. If DisplayTask had a high priority, every redraw would delay input and motion handling (Fault experiment 2). If any priority-3 task stopped blocking, all lower tasks would starve (Fault experiment 1).

Task states

Each task is **Running** only while it executes its short body (reading a sensor, drawing a frame). It is **Blocked** while it waits in `vTaskDelay()`, `vTaskDelayUntil()`, `xQueueReceive()` or `xEventGroupWaitBits()`; in this state it uses no CPU and cannot be selected by the scheduler. It is **Ready** when its delay has expired or its data has arrived but a higher-priority task is still running. Example observed in Part III: TaskA (priority 1) and TaskB (priority 2) woke on the same tick; TaskB always printed first while TaskA waited in Ready. When every task is Blocked, the idle task runs and sleeps with `__WFI()`. **Suspended** and **Deleted** are not used in this design: no task needs to be paused indefinitely or destroyed, and all run for the life of the system.

vTaskDelayUntil() versus vTaskDelay()

`vTaskDelay(2000)` blocks for 2000 ms counted from the moment it is called, which is after the sensors have been read. The real period is therefore 2000 ms plus the read time plus any time spent waiting in Ready, and this error accumulates every cycle, so samples drift later and later. `vTaskDelayUntil(&lastWakeTime, 2000)` blocks until an absolute time: the previous wake time plus 2000 ms. It then advances `lastWakeTime` by exactly one period, so the task wakes at 0, 2, 4, 6 s regardless of how long each read took (as long as the work fits in the period). SensorTask uses it because sampling should be evenly spaced; the DHT22 read alone takes several milliseconds, which would otherwise add drift on every cycle.

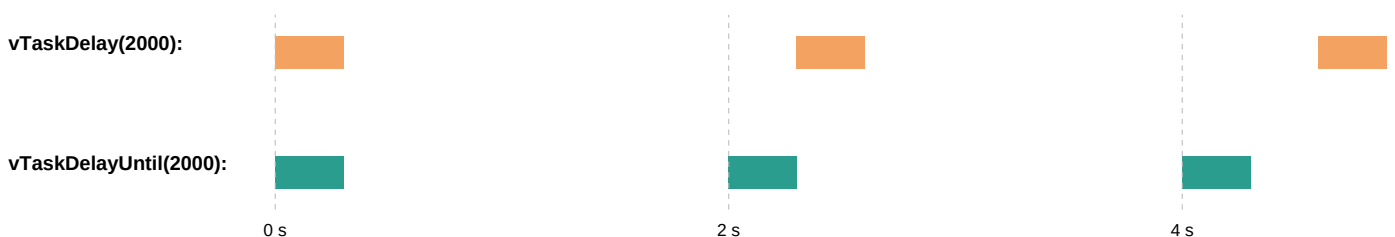


Figure 4. Wake times with a 300 ms read: `vTaskDelay` drifts by the read time each cycle; `vTaskDelayUntil` stays on a fixed 2 s grid.

Communication: queues

Queue	Producer to consumer	Item	Length / write mode
sensorToDisplayQueue	SensorTask to DisplayTask	SensorData	1 / xQueueOverwrite
sensorToAlarmQueue	SensorTask to AlarmTask	SensorData	1 / xQueueOverwrite
displayModeQueue	InputTask to DisplayTask	DisplayMode	1 / xQueueOverwrite

SensorData carries temperature, humidity, lightLevel, motionDetected and a `dhtValid` flag. A queue is used instead of global variables because `xQueueOverwrite` and `xQueueReceive` copy the whole structure atomically, so a consumer can never read a half-updated reading (for example a new temperature with an old humidity), and

the consumer is unblocked by the kernel the moment new data arrives instead of polling. Length 1 with overwrite means consumers always get the newest reading and never process a backlog of stale ones. Two sensor queues are needed because receiving from a queue removes the item: with one queue, DisplayTask and AlarmTask would steal readings from each other.

Synchronization: event group

Bit	Name	Producer (set / clear)	Consumer	Meaning
0	EVENT_ACTIVE	StateTask sets on ACTIVE, clears on INACTIVE	DisplayTask, InputTask, AlarmTask	System is ACTIVE
1	EVENT_MOTION	MotionTask sets when PIR is high	StateTask (cleared on exit of WaitBits)	Motion occurred since last check
2	EVENT_ALARM	AlarmTask sets/clears on state change	DisplayTask	Alarm active (OLED banner)
3	EVENT_PIR_LEVEL	MotionTask sets/clears every poll	SensorTask	Current PIR level for SensorData

The event group broadcasts system state to several tasks at once, which a queue cannot do. EVENT_MOTION is an *event* that is consumed exactly once (StateTask clears it with xClearOnExit = pdTRUE), while EVENT_ACTIVE and EVENT_PIR_LEVEL are *levels* that many tasks read. DisplayTask blocks on EVENT_ACTIVE while INACTIVE, which is how display work is eliminated rather than just hidden.

Shared-resource protection: mutex

Resource: USART1, the diagnostic serial port. **Competing tasks:** all six tasks log to it (for example InputTask page changes, StateTask state changes, AlarmTask alarm transitions and MotionTask motion edges can all happen within the same second). **Failure prevented:** HAL_UART_Transmit sends one character at a time; if the scheduler switched tasks during a transmission, or between the three Log() calls that build one line, characters from two tasks would mix into unreadable output such as [Input[StateTask] SystemTask] Page. serialMutex is a **recursive** mutex, so Log_Begin() can lock once for a whole multi-part line while each Log() inside it locks again safely. A mutex (not a binary semaphore) is used because it provides priority inheritance: if DisplayTask (priority 1) holds it while a priority-3 task waits, DisplayTask is temporarily raised so it releases the port quickly. The encoder ISR does not log and does not call FreeRTOS; it only changes a counter that InputTask reads and clears inside a short critical section, which is the other place a race could occur.

4. Implementation

Entry point. main() enables the vector table at FLASH_BASE, calls HAL_Init(), initializes the LED, UART, sensors, encoder and PIR, then calls app_main(). app_main() creates the RTOS objects, creates the six tasks with explicit priorities and 256-word stacks, and starts the scheduler. Any creation failure is reported on the UART and halts.

DHT22 driver. The single-wire protocol needs microsecond timing, so the driver uses the Cortex-M DWT cycle counter instead of a hardware timer. The start pulse (1.2 ms low) is sent with the pin as a push-pull output; the pin is then switched to input with pull-up, because on the STM32F1 the internal pull-up only works in input mode (the first open-drain version failed every read for this reason). The ~5 ms reply is read inside a critical section so no interrupt or task switch distorts bit timing. Every wait loop has both a time limit and an iteration limit, so the driver can never hang. The checksum is verified; a failed read sets dhtValid = false and the last good values are kept, so the alarm never reacts to garbage.

Photoresistor. ADC1 channel 0 is read with one software-triggered, polled conversion (10 ms timeout). The module's output voltage rises in darkness, so the 12-bit value is inverted: $\text{light} = (4095 - \text{raw}) \times 100 / 4095$. This is a documented relative level, not lux; the direction was verified (brighter slider gives a higher percentage).

OLED. A small SSD1306 driver keeps a 1 KB frame buffer in RAM, draws text with a 5x7 font (scaled x2 for the reading) and sends the frame over I2C1. Only DisplayTask calls it, which removes the need for a display lock. DisplayTask redraws only when data, the page or the alarm flag changes.

Encoder. The CLK pin triggers EXTI4 on a falling edge. The ISR reads DT to get the direction and adds +1 or -1 to a volatile counter. InputTask reads and clears the counter atomically every 50 ms and applies nextDisplayMode() or previousDisplayMode(), so all page logic stays outside interrupt context and is unit-tested.

Alarm. `evaluateTemperature()` in `logic.c` returns `NORMAL`, `LOW_TEMPERATURE` or `HIGH_TEMPERATURE` (18 C and 30 C themselves are normal). `AlarmTask` applies it to each valid reading and drives the buzzer with 1 kHz PWM on TIM2. It only touches the hardware when the state changes, and forces the buzzer off while `INACTIVE`.

State machine. `evaluateSystemState(current, motion, msSinceLastMotion, timeout)` is a pure function. `StateTask` waits on `EVENT_MOTION` with a 250 ms timeout, so it reacts to motion immediately and still checks the 15 s timeout regularly without busy-waiting.

C instead of C++. The laboratory examples use `enum` class and `.cpp` files. The project uses C with `typedef` enums (`DisplayMode`, `AlarmState`, `SystemState`) because the STM32Cube HAL and CubeMX-generated code are C, and the same type safety for page and state values is achieved with named enumerators instead of numeric constants.

5. Verification and Testing

Unit tests

The 13 tests run on the PC (`pio test -e native`, Unity framework) against `logic.c` only, so they need neither the STM32 nor Wokwi. Result: **13 test cases, 13 succeeded** (`docs/unit-test-results.txt`).

Area	Test	Checks
Alarm	below lower limit (17.9 C)	<code>LOW_TEMPERATURE</code>
Alarm	exactly lower limit (18.0 C)	<code>NORMAL</code> (boundary is inclusive)
Alarm	normal value (25.0 C)	<code>NORMAL</code>
Alarm	exactly upper limit (30.0 C)	<code>NORMAL</code> (boundary is inclusive)
Alarm	above upper limit (30.1 C)	<code>HIGH_TEMPERATURE</code>
Navigation	next from Temperature	Humidity
Navigation	next from Motion	Temperature (forward wraparound)
Navigation	previous from Humidity	Temperature
Navigation	previous from Temperature	Motion (reverse wraparound)
State	<code>ACTIVE</code> , no motion, 10 s elapsed	stays <code>ACTIVE</code>
State	<code>ACTIVE</code> , no motion, 15 s elapsed	becomes <code>INACTIVE</code>
State	<code>INACTIVE</code> , no motion	stays <code>INACTIVE</code>
State	<code>INACTIVE</code> , motion	becomes <code>ACTIVE</code>

Wokwi functional verification

ID	Input / stimulus	Expected	Actual (observed)	Result
FT-01	DHT22 temperature slider 25.0 to 43.7 to 58.6 C	Displayed temperature updates	Serial and OLED showed 43.7 C, then 58.6 C within one 2 s cycle	PASS
FT-02	DHT22 humidity slider 50.0 to 73.0 to 56.5 %	Displayed humidity updates	Humidity line and page followed each value	PASS
FT-03	Photoresistor lux slider up and down	Light value changes	75 % at default; brighter gave a higher %, darker a lower %	PASS
FT-04	Encoder clockwise arrow	Next page	Temperature, Humidity, Light, Motion, Temperature; log '[InputTask] Page: ...'	PASS
FT-05	Encoder counterclockwise arrow	Previous page	Temperature to Motion (wraparound), then Light	PASS
FT-06	Temperature set to 50.1 C	Alarm activates	'Alarm state: <code>HIGH_TEMPERATURE</code> ', buzzer sounded, OLED showed '! TEMP ALARM'	PASS
FT-07	Temperature returned to 24.6 C	Alarm stops	'Alarm state: <code>NORMAL</code> ', buzzer silent, banner removed	PASS
FT-08	Simulate motion while <code>ACTIVE</code>	System is <code>ACTIVE</code>	'[MotionTask] Motion detected'; stayed <code>ACTIVE</code> ; 15 s timer restarted	PASS

ID	Input / stimulus	Expected	Actual (observed)	Result
FT-09	No motion for 15 s	System becomes INACTIVE	'[StateTask] System INACTIVE', '[DisplayTask] OLED off (INACTIVE)', screen blank	PASS
FT-10	Simulate motion while INACTIVE	Returns to ACTIVE	'Motion detected', 'System ACTIVE', 'OLED on (ACTIVE)'; screen showed 25.0 C again	PASS

Additional checks: while INACTIVE, encoder turns and a temperature above 30 C produced no page change and no buzzer, as required. A single 'DHT22 read failed' appears at power-up because the sensor model is read before it has settled; later reads succeed and the failed read is ignored by the alarm.

Deliberate FreeRTOS fault experiments

Each experiment was applied alone, built, run and observed, then reverted with git checkout before the next one.

	Change	Observation	Explanation
1. Remove blocking	vTaskDelay() removed from MotionTask (priority 3)	Only '[MotionTask] EXPERIMENT 1' was printed. The OLED stayed on '...', no sensor data, no '[StateTask] System ACTIVE', no alarm; the system never went INACTIVE.	MotionTask never leaves the Ready state and is the highest-priority Ready task, so every other task starves and the idle task never runs (CPU at 100 %, no __WFI sleep). In this port task switches happen at blocking calls, so even equal-priority tasks starved; with a standard preemptive port, InputTask and StateTask would time-slice with it, but Display, Sensor and Alarm (priorities 1-2) would still starve.
2. Raise priority	DisplayTask 1 to 4 (highest)	No visible failure. Encoder, PIR and alarm (HIGH_TEMPERATURE then NORMAL at 24.6 C) still responded.	DisplayTask is Blocked most of the time, and priority only matters while a task is Ready. During each I2C redraw, however, it now outranks InputTask and MotionTask, so urgent input can wait behind a non-urgent redraw. With a busier display this becomes visible lag, which is why it was restored to priority 1.
3. Remove mutex	MutexUsable() forced to return 0	No interleaved lines seen while turning the encoder, triggering the PIR and crossing 30 C together.	In this compatibility port a task switch only occurs at a blocking call, and polled HAL_UART_Transmit() never blocks, so a line is never interrupted. The port is still unsafe to rely on: with a standard preemptive port the tick can switch tasks mid-line and produce mixed output. The mutex makes correctness independent of the scheduler, so it was restored.

6. Static Code Analysis

pio check -e bluepill_f103c8 (cppcheck) was run on the project sources. Both reports are kept in the repository (docs/pio-check-report.txt and docs/pio-check-report-after.txt).

Run	High	Medium	Low
Initial	0	0	58
Final	0	0	6

Finding	File / line	Count	Cause	Resolution
unusedFunction	All task files, drivers, logic.c, app.c, stm32f1xx_it.c, stm32f1xx_hal_msp.c, timebase	51	False positive. cppcheck checks each file separately and cannot see calls through function pointers (xTaskCreate), the interrupt vector table (EXTI4_IRQHandler, TIM4_IRQHandler) or HAL/FreeRTOS callbacks (HAL_*_MspInit, vApplicationIdleHook).	No code change. Suppressed with --suppress=unusedFunction in platformio.ini, with the justification as a comment.
unsignedLessThanZero	dht22.c:68 (WaitLevel); later also DelayUs	1	The loop condition mixed a volatile register read with unsigned subtraction. The expression was rewritten with a named 'elapsed' variable; the warning followed the new expression, confirming a false positive on a wrap-safe elapsed-time check (there is no sign comparison).	Code clarified, then an inline cppcheck-suppress with justification (--inline-suppr enabled).

Finding	File / line	Count	Cause	Resolution
constParameterP ointer	rtos_hooks.c:20, :32 (pcTaskName, htim)	2	The signatures of vApplicationStackOverflowHook and HAL_TIM_PeriodElapsedCallback are fixed by FreeRTOS and HAL.	Kept: changing them would no longer match the API declarations.
constParameterP ointer	stm32f1xx_hal_msp.c: 89, 124, 155, 195	4	CubeMX-generated MSP callbacks with fixed HAL signatures.	Kept: generated code, fixed API.

Interpretation: no finding indicated a real defect; the value of the run was confirming that, documenting why each remaining warning is intentional, and making the DHT22 timing code clearer.

7. Engineering Discussion

Debugging: running FreeRTOS in Wokwi

The firmware built and printed its start-up messages, but no task ever ran after vTaskStartScheduler(), and the simulator slowed to about 6 % speed. Because a silent lock-up gives no information, the first step was to make failures visible: configASSERT was mapped to vAssertCalled(), which writes the file and line directly to the USART1 registers (HAL printing is not safe at that point), and a HardFault handler was added that reports the stacked PC. HAL was also moved to TIM4 with STM32CubeMX so that SysTick belongs only to FreeRTOS.

Step	Configuration	Evidence	Conclusion
1	FreeRTOS V11 ARM_CM3 port	ASSERT at port.c:336	The check writes an NVIC priority and reads it back as 0: the Wokwi STM32F103 model does not implement priority bits, so BASEPRI masking cannot work.
2	V11 ARM_CM0 port (PRIMASK)	ASSERT at port.c:901 (Incorrect SVC call)	The SVC number used to start the first task is not recovered correctly from the exception frame in the simulator.
3	V10.6.2 ARM_CM0 port	Tasks printed once. An idle-hook probe showed the tick counting 233, 465, 696, 928, then stopping just before tick 1000.	The tick works; the CPU locks when the first delayed task is resumed through an interrupt-driven (PendSV) context switch. Turning preemption off did not help.
4	Compatibility port (TIM3 tick, Thread-mode switching), adapted from classmate Ni-ear	Task A and Task B repeat every second	Avoids SysTick/SVC/PendSV entirely.
5	Same port after adding the DHT22	System froze after a few readings, especially when slider values changed	Two port defects, fixed in this project (below).

Fix 1, interrupt masking: the port still masked interrupts with BASEPRI, which (step 1) has no effect in Wokwi, so the TIM3 tick could interrupt kernel critical sections and corrupt task lists. portmacro.h was changed to use PRIMASK (cpsid i / cpsie i). **Fix 2, critical nesting:** once masking really worked, the system froze permanently. FreeRTOS can yield from inside a critical section, and the port kept one shared nesting counter, so the next task inherited a wrong count and interrupts stayed disabled forever. The context-switch routine now saves and restores the counter in each task's context frame, as the official ports do. After both fixes the system ran continuously through all functional tests.

Other problems encountered

Problem	Cause	Fix
'expected declaration at end of input'	Missing closing brace in main()	Added the brace
Multiple definition of SysTick_Handler	Both main.c and the port defined it	One owner per handler (later none needed)
Circular include error in FreeRTOSConfig.h	stm32f1xx.h pulled in CubeMX headers that include FreeRTOS	Declared only SystemCoreClock with stdint.h
Every DHT22 read failed	Open-drain output pull-up does not work on STM32F1	Output for the start pulse, then input with pull-up

Problem	Cause	Fix
Buzzer on PB0 conflicted with the RTOS tick	PB0 is a TIM3 channel	Moved to PA2 (TIM2_CH3)
Native unit tests could not build	No host gcc on Windows	Installed WinLibs GCC with winget

Trade-offs and alternative approaches

Polling vs interrupts for the PIR: 100 ms polling was chosen because motion is slow compared with the poll rate and it avoids another ISR; an EXTI interrupt with a task notification would give lower latency on hardware. **Task notification vs event group:** a notification would be lighter for the single motion-to-StateTask signal, but the ACTIVE state must be read by three tasks, which is a natural event-group use. **Latest-value queues vs deeper queues:** overwriting loses intermediate readings, which is acceptable because only the current room condition matters. **Tick rate:** the compatibility port uses a 20 Hz tick (50 ms resolution) to keep the simulator fast; this is enough for 50 ms to 2 s periods but not for fine timing. **Alternative board:** a Wokwi board with a better-supported core, or the Arduino core, might have run the stock port, but the laboratory requires STM32Cube on the Blue Pill.

Limitations

The scheduling port is simulator-specific: the tick does not preempt a running task, switches occur at blocking calls and from the idle hook, and FromISR APIs are not used. On a physical Blue Pill the standard ARM_CM3 port with a 1 kHz SysTick should be used, and Experiments 1 and 3 would then show time-slicing and interleaved output as described. The light value is uncalibrated. The DHT22 read disables interrupts for about 5 ms. The 15 s timeout is deliberately short for testing. On hardware, sensor noise, contact bounce on the encoder and real UART timing would also need to be handled.

Requirements traceability matrix

Requirement	Implementation	Verification
FR-01 Temperature	SensorTask, dht22.c	FT-01
FR-02 Humidity	SensorTask, dht22.c	FT-02
FR-03 Ambient light	SensorTask, ldr.c	FT-03
FR-04 Motion detection	MotionTask, EVENT_MOTION / EVENT_PIR_LEVEL	FT-08, FT-10
FR-05 OLED display	DisplayTask, ssd1306.c, sensorToDisplayQueue	FT-01 to FT-05
FR-06 Encoder navigation	InputTask, nextDisplayMode(), previousDisplayMode(), displayModeQueue	4 navigation unit tests + FT-04, FT-05
FR-07 Temperature alarm	AlarmTask, evaluateTemperature(), buzzer.c, EVENT_ALARM	5 alarm unit tests + FT-06, FT-07
FR-08 Activity state	StateTask, EVENT_ACTIVE	State unit tests + FT-08 to FT-10
FR-09 Automatic inactivity	StateTask, evaluateSystemState(), 15 s timeout	ACTIVE/timeout unit test + FT-09
FR-10 Automatic reactivation	MotionTask to StateTask via EVENT_MOTION	INACTIVE/motion unit test + FT-10

8. Conclusion

The system meets all ten functional requirements with six tasks whose priorities follow their latency needs, three latest-value queues for data, an event group for system state, a mutex for the shared serial port and vTaskDelayUntil() for drift-free sampling. Separating decision logic into logic.c made 13 meaningful unit tests possible without hardware, and static analysis confirmed there were no real defects.

The most useful lesson came from the failures. A task that stops blocking takes over the processor, a priority only matters while a task is Ready, and correctness should not depend on the scheduler happening to switch at safe moments. Debugging the port showed that turning silent hangs into printed asserts and measurements (the tick counter) is far faster than guessing, and that a simulator is itself a platform with its own limits that must be documented.

Future improvements: validate on a physical Blue Pill with the standard ARM_CM3 port; drive the encoder and PIR with task notifications from ISRs; use interrupt- or DMA-driven UART logging; calibrate the light sensor; and

show more status on the OLED.

References and acknowledgments

BCA182 Laboratory Activity No. 1 specification, Asst. Prof. Paul Rodolf P. Castor, MSU-IIT. FreeRTOS Kernel V10.3.1, MIT license, github.com/FreeRTOS/FreeRTOS-Kernel. Wokwi compatibility port ([lib/FreeRTOS/portable/GCC/ARM_CM3](https://github.com/FreeRTOS/portable/GCC/ARM_CM3)) adapted from classmate Ni-ear, github.com/Ni-ear/bca182-freertos-multisensor; the PRIMASK masking and per-task critical-nesting fixes were added in this project. STMicroelectronics STM32CubeF1 HAL and STM32CubeMX (HAL configuration, MSP, interrupt and TIM4 timebase files). PlatformIO ststm32 platform, Unity test framework and cppcheck. Wokwi simulator documentation (docs.wokwi.com). Classic public-domain 5x7 ASCII font. Claude (Anthropic) was used as an assistant for debugging guidance, code drafting and documentation; all code was built, run and verified by the author.